Mini Project 8 — Menetapkan Feature Schema untuk Keyword Spotting

Project 8 melanjutkan langsung MP7. Mahasiswa tidak merekam ulang audio dan tidak melakukan preprocessing ulang. Skrip membaca frame baku MP7, menghitung RMS, ZCR, centroid, bandwidth, roll-off, log-Mel, 13 MFCC, dan pitch sederhana. Hasil frame-level disimpan bersama vektor 39 dimensi dan konfigurasi fitur. Status proyek diubah menjadi ready_for_mp09.

Tabel 8.12. Input dan output Mini Project 8.

Jenis	Path	Peran
Status input	project_status.json = ready_for_mp08	Validasi serial
Frame input	features/intermediate/mp07_preprocessed_frames.npz	Input utama fitur
Config input	config/preprocessing_config.json	Konsistensi fs/frame/hop
Config output	config/feature_config.json	Kontrak fitur seluruh dataset
Frame features	features/intermediate/mp08_frame_features.npz	Log-Mel, MFCC, scalar features
Sample vector	features/intermediate/mp08_sample_feature_vector.npz	Baseline 39-D
Metrik	results/metrics/mp08_feature_metrics.json	Validasi dimensi/numerik
Status output	project_status.json = ready_for_mp09	Handoff serial

Prosedur Step by Step

Langkah 1 — Validasi artefak MP7

Buka terminal pada root `audio_ai_embedded_project`. Pastikan project_status.json bernilai ready_for_mp08.

```
python -c "import json; s=json.load(open('project_status.json')); print(s['status']); assert s['status']=='ready_for_mp08'"
```

Langkah 2 — Buat skrip Mini Project 8

Buat file `mini_project_08.py` pada root proyek dan salin kode Python di bawah.

Langkah 3 — Jalankan ekstraksi fitur

Jalankan `python mini_project_08.py`. Skrip menghitung seluruh fitur dan menyimpan config, NPZ, metrik, serta dua gambar.

```
python mini_project_08.py
```

Langkah 4 — Periksa visualisasi

Buka `mp08_logmel_mfcc.png` dan `mp08_scalar_features.png`. Pastikan struktur time-frequency terlihat dan tidak ada NaN/Inf.

Langkah 5 — Periksa vektor 39-D

Buka metrik JSON atau jalankan perintah pengecekan untuk memastikan vector_dim=39 dan feature_names berjumlah 39.

```
python -c "import numpy as np; z=np.load('features/intermediate/mp08_sample_feature_vector.npz',allow_pickle=True); print(len(z['values']),
len(z['names'])); assert len(z['values'])==39"
```

Langkah 6 — Validasi handoff MP9

Pastikan status berubah menjadi ready_for_mp09 serta feature_config dan sample vector tercatat di project_status.json.

Kode Python Mini Project 8

```
from pathlib import Path
import json
import numpy as np
import matplotlib.pyplot as plt
from scipy.fft import dct

ROOT = Path('.')
STATUS = ROOT / 'project_status.json'
status = json.loads(STATUS.read_text(encoding='utf-8'))
assert status['status'] == 'ready_for_mp08', status['status']

frames_path = ROOT / status['mp08_input_frames']
pre_cfg_path = ROOT / status['preprocessing_config']
assert frames_path.exists() and pre_cfg_path.exists()

z = np.load(frames_path, allow_pickle=True)
frames = np.asarray(z['frames'], dtype=np.float64)
fs = int(z['fs'])
frame_len = int(z['frame_length'])
hop_len = int(z['hop_length'])
assert frames.ndim == 2 and frames.shape[1] == frame_len
assert np.isfinite(frames).all()

pre_cfg = json.loads(pre_cfg_path.read_text(encoding='utf-8'))
assert int(pre_cfg['sampling_rate_hz']) == fs

# ---------- 1) Basic frame features ----------
rms = np.sqrt(np.mean(frames**2, axis=1) + 1e-12)
zcr = np.mean(np.abs(np.diff(np.signbit(frames), axis=1)), axis=1)

freq = np.fft.rfftfreq(frame_len, d=1.0/fs)
X = np.fft.rfft(frames, axis=1)
power = (np.abs(X) ** 2) + 1e-15
psum = np.sum(power, axis=1) + 1e-15
centroid = np.sum(power * freq[None, :], axis=1) / psum
bandwidth = np.sqrt(np.sum(power * (freq[None, :] - centroid[:, None])**2,
                    axis=1) / psum)

cum = np.cumsum(power, axis=1)
rolloff = np.empty(len(frames), dtype=np.float64)
for i in range(len(frames)):
    idx = int(np.searchsorted(cum[i], 0.85 * cum[i, -1]))
    rolloff[i] = freq[min(idx, len(freq)-1)]

# ---------- 2) Mel filterbank, log-Mel, MFCC ----------
def hz_to_mel(f):
    return 2595.0 * np.log10(1.0 + np.asarray(f) / 700.0)

def mel_to_hz(m):
    return 700.0 * (10.0**(np.asarray(m) / 2595.0) - 1.0)

def make_mel_filterbank(n_fft, fs, n_mels=26, fmin=80.0, fmax=7600.0):
    n_bins = n_fft // 2 + 1
    mel_pts = np.linspace(hz_to_mel(fmin), hz_to_mel(fmax), n_mels + 2)
    hz_pts = mel_to_hz(mel_pts)
    bins = np.floor((n_fft + 1) * hz_pts / fs).astype(int)
    bins = np.clip(bins, 0, n_bins - 1)
    fb = np.zeros((n_mels, n_bins), dtype=np.float64)
    for m in range(1, n_mels + 1):
        left, center, right = bins[m-1], bins[m], bins[m+1]
        center = max(center, left + 1)
        right = max(right, center + 1)
        right = min(right, n_bins - 1)
        for k in range(left, min(center, n_bins)):
            fb[m-1, k] = (k - left) / max(1, center - left)
        for k in range(center, min(right + 1, n_bins)):
            fb[m-1, k] = (right - k) / max(1, right - center)
    return fb

n_mels = 26
n_mfcc = 13
fmin = 80.0
fmax = min(7600.0, 0.95 * fs/2)
fb = make_mel_filterbank(frame_len, fs, n_mels, fmin, fmax)
mel_energy = np.maximum(power @ fb.T, 1e-12)
```

```
log_mel = np.log(mel_energy)
mfcc = dct(log_mel, type=2, axis=1, norm='ortho')[:, :n_mfcc]

# ---------- 3) Pitch via autocorrelation ----------
pitch_min_hz = 80.0
pitch_max_hz = 350.0
lag_min = max(1, int(fs / pitch_max_hz))
lag_max = min(frame_len - 2, int(fs / pitch_min_hz))
pitch_hz = np.zeros(len(frames), dtype=np.float64)
pitch_conf = np.zeros(len(frames), dtype=np.float64)
energy_floor = float(np.percentile(rms, 25))

for i, fr in enumerate(frames):
    if rms[i] <= energy_floor:
        continue
    ac = np.correlate(fr, fr, mode='full')[frame_len-1:]
    if ac[0] <= 1e-12:
        continue
    ac = ac / ac[0]
    search = ac[lag_min:lag_max+1]
    if len(search) == 0:
        continue
    lag = int(np.argmax(search)) + lag_min
    conf = float(ac[lag])
    pitch_conf[i] = conf
    if conf >= 0.25:
        pitch_hz[i] = fs / lag

voiced = pitch_hz > 0
voiced_ratio = float(np.mean(voiced))

# ---------- 4) Aggregate to fixed 39-D vector ----------
feature_names = []
feature_values = []

def add_mean_std(prefix, arr, mask=None):
    a = np.asarray(arr, dtype=np.float64)
    if mask is not None:
        a = a[np.asarray(mask, dtype=bool)]
    if a.size == 0:
        mu, sd = 0.0, 0.0
    else:
        mu, sd = float(np.mean(a)), float(np.std(a))
    feature_names.extend([prefix + '_mean', prefix + '_std'])
    feature_values.extend([mu, sd])

for j in range(n_mfcc):
    add_mean_std(f'mfcc{j+1:02d}', mfcc[:, j])
add_mean_std('rms', rms)
add_mean_std('zcr', zcr)
add_mean_std('centroid_hz', centroid)
add_mean_std('bandwidth_hz', bandwidth)
add_mean_std('rolloff85_hz', rolloff)
add_mean_std('f0_hz', pitch_hz, voiced)
feature_names.append('voiced_ratio')
feature_values.append(voiced_ratio)

feature_values = np.asarray(feature_values, dtype=np.float32)
feature_names = np.asarray(feature_names, dtype=object)
assert feature_values.shape == (39,), feature_values.shape
assert len(feature_names) == 39
assert np.isfinite(feature_values).all()

# ---------- 5) Save artefacts ----------
config_dir = ROOT / 'config'; config_dir.mkdir(parents=True, exist_ok=True)
feat_dir = ROOT / 'features/intermediate'; feat_dir.mkdir(parents=True, exist_ok=True)
metric_dir = ROOT / 'results/metrics'; metric_dir.mkdir(parents=True, exist_ok=True)
fig_dir = ROOT / 'results/figures'; fig_dir.mkdir(parents=True, exist_ok=True)

feature_cfg = {
    'sampling_rate_hz': fs,
    'frame_length_samples': frame_len,
    'hop_length_samples': hop_len,
    'input_frames_source': str(frames_path.relative_to(ROOT)),
    'power_spectrum': 'abs(rfft(frame))**2',
    'mel': {'n_mels': n_mels, 'fmin_hz': fmin, 'fmax_hz': fmax,
            'scale': '2595*log10(1+f/700)', 'log': 'natural'},
    'mfcc': {'n_mfcc': n_mfcc, 'dct_type': 2, 'norm': 'ortho',
             'delta': False, 'delta_delta': False},
    'spectral': {'centroid': True, 'bandwidth': True, 'rolloff_fraction': 0.85},
    'temporal': {'rms': True, 'zcr': True},
    'pitch': {'method': 'autocorrelation', 'min_hz': pitch_min_hz,
              'max_hz': pitch_max_hz, 'confidence_threshold': 0.25},
    'aggregation': {'statistics': ['mean', 'std'], 'f0_only_voiced': True,
                    'append_voiced_ratio': True},
    'sample_vector_dim': 39,
    'scaling': 'NONE_HERE__fit_only_on_training_set_in_later_chapter'
}
(config_dir / 'feature_config.json').write_text(
    json.dumps(feature_cfg, indent=2), encoding='utf-8')
```

```
np.savez(feat_dir / 'mp08_frame_features.npz',
        fs=np.int32(fs),
        rms=rms.astype(np.float32), zcr=zcr.astype(np.float32),
        centroid_hz=centroid.astype(np.float32),
        bandwidth_hz=bandwidth.astype(np.float32),
        rolloff85_hz=rolloff.astype(np.float32),
        pitch_hz=pitch_hz.astype(np.float32),
        pitch_confidence=pitch_conf.astype(np.float32),
        log_mel=log_mel.astype(np.float32),
        mfcc=mfcc.astype(np.float32),
        mel_filterbank=fb.astype(np.float32))

np.savez(feat_dir / 'mp08_sample_feature_vector.npz',
        sample_id='nyala_001', label='nyala',
        values=feature_values, names=feature_names)

metrics = {
   'n_frames': int(len(frames)),
   'frame_feature_shapes': {'log_mel': list(log_mel.shape), 'mfcc': list(mfcc.shape)},
   'sample_vector_dim': int(len(feature_values)),
   'voiced_ratio': voiced_ratio,
   'mean_f0_hz_voiced': float(np.mean(pitch_hz[voiced])) if np.any(voiced) else 0.0,
   'std_f0_hz_voiced': float(np.std(pitch_hz[voiced])) if np.any(voiced) else 0.0,
   'nan_or_inf_count': int(np.size(feature_values) - np.isfinite(feature_values).sum()),
   'feature_names': feature_names.tolist()
}
(metric_dir / 'mp08_feature_metrics.json').write_text(
   json.dumps(metrics, indent=2), encoding='utf-8')

# ---------- 6) Figures ----------
fig, axs = plt.subplots(2, 1, figsize=(10, 6.5), sharex=True)
im0 = axs[0].imshow(log_mel.T, aspect='auto', origin='lower', interpolation='nearest')
axs[0].set_ylabel('Mel bin'); axs[0].set_title('Log-Mel')
fig.colorbar(im0, ax=axs[0], fraction=.025, pad=.02)
im1 = axs[1].imshow(mfcc.T, aspect='auto', origin='lower', interpolation='nearest')
axs[1].set_ylabel('MFCC'); axs[1].set_xlabel('Frame'); axs[1].set_title('13 MFCC')
fig.colorbar(im1, ax=axs[1], fraction=.025, pad=.02)
fig.tight_layout(); fig.savefig(fig_dir/'mp08_logmel_mfcc.png', dpi=180); plt.close(fig)

fig, axs = plt.subplots(3, 1, figsize=(10, 7.2), sharex=True)
axs[0].plot(rms, label='RMS'); axs[0].plot(zcr, label='ZCR'); axs[0].legend(); axs[0].grid(alpha=.2)
axs[1].plot(centroid, label='centroid'); axs[1].plot(bandwidth, label='bandwidth'); axs[1].plot(rolloff, label='rolloff85'); axs[1].set_ylabel('Hz');
axs[1].legend(); axs[1].grid(alpha=.2)
axs[2].plot(pitch_hz, label='F0'); axs[2].set_ylabel('Hz'); axs[2].set_xlabel('Frame'); axs[2].legend(); axs[2].grid(alpha=.2)
fig.tight_layout(); fig.savefig(fig_dir/'mp08_scalar_features.png', dpi=180); plt.close(fig)

# ---------- 7) Update serial status ----------
status.update({
   'status': 'ready_for_mp09',
   'last_completed_mini_project': 8,
   'feature_config': 'config/feature_config.json',
   'frame_features': 'features/intermediate/mp08_frame_features.npz',
   'sample_feature_vector': 'features/intermediate/mp08_sample_feature_vector.npz',
   'feature_metrics': 'results/metrics/mp08_feature_metrics.json',
   'feature_schema_dim': 39,
   'mp09_input_feature_config': 'config/feature_config.json',
   'mp09_reference_feature_vector': 'features/intermediate/mp08_sample_feature_vector.npz',
   'mp09_reference_frame_features': 'features/intermediate/mp08_frame_features.npz'
})
STATUS.write_text(json.dumps(status, indent=2), encoding='utf-8')

print('Frames          :', len(frames))
print('Log-Mel shape   :', log_mel.shape)
print('MFCC shape      :', mfcc.shape)
print('Feature vector dim:', len(feature_values))
print('Voiced ratio    :', round(voiced_ratio, 3))
print('Status          : ready_for_mp09')
```

Kriteria Lulus Mini Project 8

Tabel 8.13. Checklist kelulusan dan handoff ke Mini Project 9.

Artefak / pemeriksaan	Kriteria lulus	Dipakai MP9
project_status.json	status = ready_for_mp09	Ya
feature_config.json	n_mels=26; n_mfcc=13; vector_dim=39; scaling=NONE	Ya, kontrak utama
mp08_frame_features.npz	log_mel T×26; mfcc T×13; tanpa NaN/Inf	Ya, referensi

Artefak / pemeriksaan	Kriteria lulus	Dipakai MP9
mp08_sample_feature_vector.npz	values dan names masing-masing 39	Ya, reference sample
mp08_feature_metrics.json	sample_vector_dim=39; nan_or_inf_count=0	Audit
mp08_logmel_mfcc.png	Struktur time-frequency terlihat	Audit
mp08_scalar_features.png	Kurva fitur dapat dibaca dan F0 hanya pada frame voiced	Audit

Handoff wajib ke Mini Project 9

Mini Project 9 akan memperluas running project menjadi dataset empat kelas. Semua sampel baru harus diproses dengan preprocessing_config dari MP7 dan feature_config dari MP8. Nama, urutan, dan jumlah 39 fitur tidak boleh berubah. Scaling masih belum dilakukan sampai training/validation/test split selesai.

Referensi Bab

1. Davis, S., & Mermelstein, P. Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences. IEEE Transactions on Acoustics, Speech, and Signal Processing.

2. Rabiner, L. R., & Schafer, R. W. Theory and Applications of Digital Speech Processing. Pearson.

3. Müller, M. Fundamentals of Music Processing. Springer.

4. Oppenheim, A. V., & Schafer, R. W. Discrete-Time Signal Processing. Pearson.

5. Lyons, R. G. Understanding Digital Signal Processing. Pearson.

6. Dokumentasi NumPy/SciPy untuk FFT, DCT, dan operasi numerik yang digunakan pada praktikum.